

**PROJECT: EASYSHARE
DEVELOPMENT
DOCUMENTATION**



Development Documentation

for the

EasyShare System

GLOBAL RTS

PROJECT: EASYSHARE DEVELOPMENT DOCUMENTATION



Content

1. Overview	3
2. .Project Structure	3
3. Technologies Used	3
4. Development Setup Guide	4
5. Authentication and Security	4
6. Development Guide	5
6.1. Login Module	5
6.2. Home Page Functions	5
6.3. Search	6
7. Database Interaction	6
8. File Upload & Validation	6
9. Permissions Management	7
10. Help & Admin Features	7
11. Code Standards	7

PROJECT: EASYSHARE DEVELOPMENT DOCUMENTATION



1. Overview

EasyShare is a secure, user-friendly web-based file system allowing folder organization, file uploads, permission control, and recovery. Built with Express, PostgreSQL, and EJS, it supports fine-grained access control and 2FA-secured logins

2. Project Structure

EasyShare/MainJavaCode/

```
|— node_modules/
|— public/
|   |— icon/
|   |— java/
|   |— style
|— views/
|— connect_db.env
|— package.json
|— package-lock.json
|— server.js
|— README.md
```

3. Technologies Used

- **Backend:** Express.js (Node.js), PostgreSQL
- **Frontend:** EJS, Bootstrap 5
- **Security:** CSRF, Helmet, bcrypt, speakeasy (2FA), HPP
- **File Handling:** Multer (memory storage)

- **Validation:** Joi

4. Development Setup Guide

- Clone the repository

```
git clone https://github.com/IChervenkov/EasyShare.git  
  
cd EasyShare
```

- Install dependencies

```
npm install
```

- Set up .env file with required variables
- Run backend

```
npm run dev
```

- Access on: <http://localhost:3000>

5. Authentication and Security

- **Login:** Username/password validated with *bcryptjs*.
- **2FA:** Enabled using *speakeasy* with TOTP + QR codes via *qrcode*.
- **Brute Force Protection:** 10 failed attempts trigger a 5-minute block.
- **Sessions:** Stored in memory (Redis, commented), secured with:

```
sameSite: 'strict', httpOnly: true, maxAge: 8hrs
```

- **CSRF Protection:** Enabled using *csurf* for all POST/PUT/PATCH/DELETE.

6. Development Guide

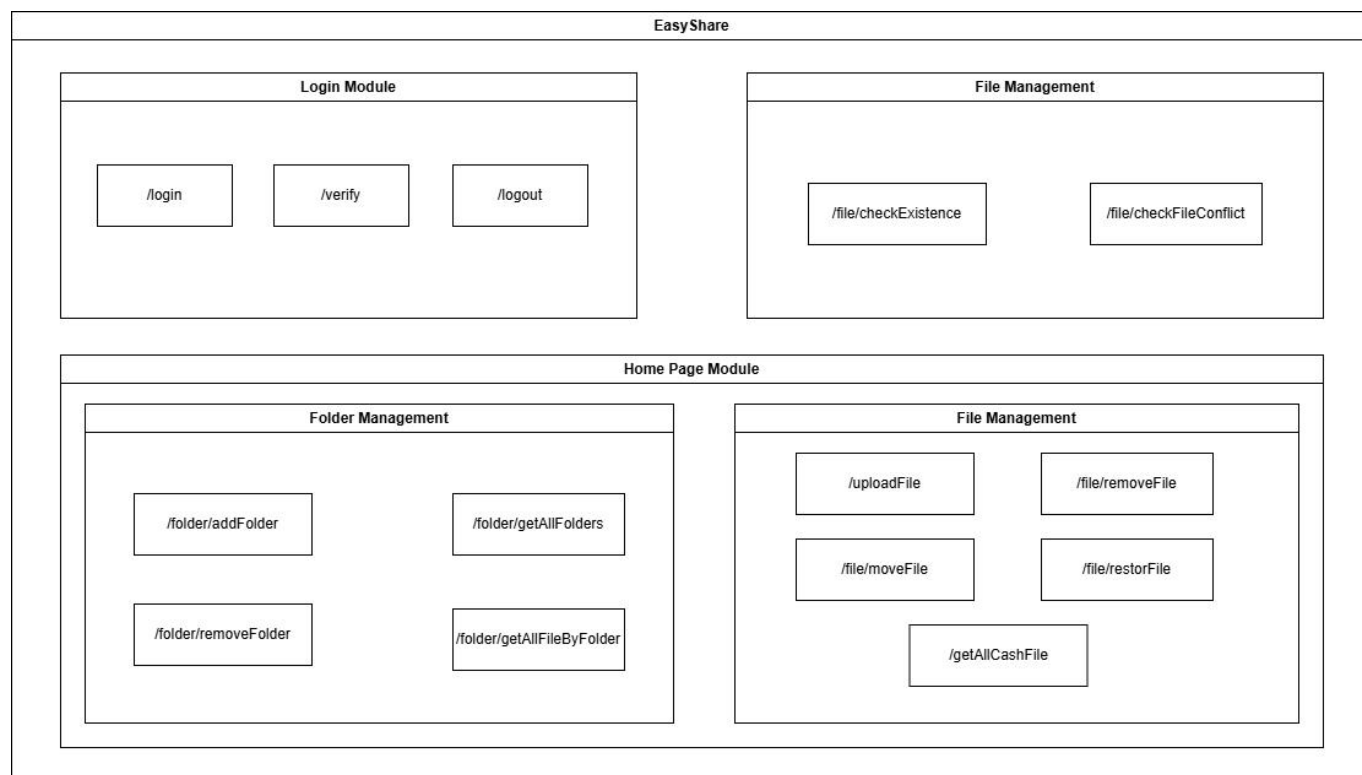


Diagram 1: Module diagram

6.1. Login Module

- `/login` – Login page
- `/verify` – 2FA code validation
- `/logout` – Destroys session

6.2. Home Page Functions

- `/folder/addFolder` – Add folders (supports full path like root/sub)
- `/folder/removeFolder` – Removes only empty folders
- `/uploadFile` – Upload files with validation and auto-override

PROJECT: EASYSHARE DEVELOPMENT DOCUMENTATION



- */file/moveFile* – Move files and subfolders
- */folder/getAllFolders* – Recursively gets folders with file count
- */folder/getAllFileByFolder* – Lists folders/files in a specific folder
- */file/removeFile* – Hard-delete files
- */file/restorFile* – Restore soft-deleted files (admin only)
- */getAllCashFile* – Lists all soft-deleted files

6.3. Search

- */file/checkExistence* – Checks for existing filenames
- */file/checkFileConflict* – Detects conflicts before move/restore

7. Database Interaction

- Uses *pg.Pool* with credentials from *connect_db.env*
- Uses transactions for all critical operations
- Folders/files are stored with permission and ownership metadata
- Recursive queries implemented using *WITH RECURSIVE*

8. File Upload & Validation

- Multer with *memoryStorage* max 100MB:
 - ❖ *.pdf, .doc, .docx* : max 10 MB
 - ❖ *.xlsx, .xls*: max 20 MB
 - ❖ *.csv, .jpg, .jpeg, .png* : max 5 MB
 - ❖ *.dwg*: max 100 MB

PROJECT: EASYSHARE DEVELOPMENT DOCUMENTATION



- MIME-type and extension matching
- Magic number check to verify file headers
- Supported extensions: *.doc*, *.docx*, *.xls*, *.xlsx*, *.csv*, *.pdf*, *.dwg*, *.jpg*, *.jpeg*, *.png*.
- Files stored as *BYTEA* in the database

9. Permissions Management

- **Tables:** *file_permission*, *folder_permission*
- **Permissions are recursive:** setting a folder permission cascades to subfolders and files.
- **Routes:**
 - ❖ *POST /setFileFolderPermission*
 - ❖ *POST /checkPermissionUser*
 - ❖ *DELETE /clearFileFolderPermission*
- UI can use */getAllUsers* and */getCurentUserId* for permission dialogs

10. Help & Admin Features

- **Help Page** (*/help*): View-only EJS template with system instructions
- **Monitoring:** All actions logged to *usermonitoring* table
- **Deleted Items Recovery:** Admin can list and restore soft-deleted files

11. Code Standards

- **JS style:** ES6+
- **Variables:** *camelCase*, constants: *UPPER_CASE*

PROJECT: EASYSHARE DEVELOPMENT DOCUMENTATION



- **Database access:** *async/await* with *try/catch/finally*
- **Form input:** validated using *Joi*
- **Magic strings** (e.g. paths, permissions): defined in route logic or middleware

